TP13 — Déplacement, actions et machines à états

Projet Java : RPG tactique inspiré de Fire Emblem • Niveau : progression L1 • 10 exercices

Objectif général. Faire évoluer le moteur du TP12 pour qu'une unité puisse se déplacer puis effectuer une action pendant la même activation, sans pouvoir se déplacer deux fois ni agir après avoir terminé son tour.

Point de départ. Conserver les classes et méthodes réalisées jusqu'au TP12 : Unite, Partie, EtatUnite, les équipes, les coordonnées, la distance de Manhattan (maximum 3), l'occupation des cases, deplacerUnite, attaquer, soigner, attendre, phaseTerminee et reinitialiserEtat. Ne pas recommencer le projet.

Nouvelle notion : une machine à états finis (Finite State Machine, FSM) représente un ensemble limité d'états et les transitions autorisées entre eux. Les actions du joueur déclenchent ces transitions ; les actions refusées ne changent pas l'état.

Règles de gameplay retenues pour ce TP

État	Sens	Actions possibles
DISPONIBLE	L'unité n'a pas bougé et n'a pas terminé son activation.	Se déplacer une seule fois ; attaquer ; soigner (si compatible) ; attendre.
DEPLACEE	L'unité a effectué son déplacement.	Attaquer ; soigner (si compatible) ; attendre. Aucun nouveau déplacement.
TERMINEE	L'activation est finie.	Aucune action ni déplacement.

Convention du TP. Une attaque ou un soin réussi termine l'activation. Une attente réussie termine l'activation. Le déplacement réussi place l'unité dans l'état DEPLACEE. Toute action refusée laisse position, PV et état inchangés. Une unité à 0 PV ne peut agir. La méthode de distance et les vérifications de cases du TP12 restent en vigueur. Les vérifications de portée des attaques et de cheminement autour des obstacles seront traitées dans des TP ultérieurs.

Méthode de travail. À chaque exercice : lire les consignes, réfléchir aux cas limites, coder sans solution fournie, compiler, exécuter les validations, puis faire un commit et un push une fois l'exercice validé. Recherche documentaire autorisée et encouragée.

Partie A — Concevoir les transitions

Exercice 1 — Décrire la machine à états

Travail demandé. Sur papier ou dans un commentaire court, dresser un tableau qui donne l'état de départ, l'action demandée, le résultat de l'action et l'état obtenu. Inclure au moins déplacement, attaque, soin et attente, depuis chacun des trois états.

Contrainte. Distinguer une action réussie d'une action refusée. Ne pas modifier le code du moteur dans cet exercice.

Validation. Ton tableau explique notamment : DISPONIBLE → déplacement réussi → DEPLACEE ; DEPLACEE → attaque réussie → TERMINEE ; TERMINEE → toute action → refus.

Exercice 2 — Introduire l'état DEPLACEE

Travail demandé. Faire évoluer EtatUnite pour représenter les trois états du tableau. Vérifier que les unités commencent toujours DISPONIBLE lors de la création d'une Partie.

Contrainte. Conserver la compatibilité avec les noms DISPONIBLE et TERMINEE ; ne pas remplacer l'énumération par des chaînes de caractères.

Validation. Les trois états peuvent être consultés et affichés. Une nouvelle Partie initialise toujours toutes ses unités à DISPONIBLE.

Notion à découvrir : Documentation Java : types énumérés (enum).

Exercice 3 — Exprimer les autorisations

Travail demandé. Faire évoluer la logique de Partie pour distinguer les unités qui peuvent encore effectuer une action de celles qui peuvent encore se déplacer. Identifier les méthodes existantes à adapter et, si utile, prévoir une opération de vérification dédiée au déplacement.

Contrainte. Ne pas recopier la condition « PV > 0 » dans chaque action. Les tests d'état doivent être centralisés et s'appuyer sur les états réels des unités.

Validation. Une unité DISPONIBLE et vivante peut agir et se déplacer ; une unité DEPLACEE et vivante peut agir mais ne peut plus se déplacer ; une unité TERMINEE ou à 0 PV ne peut faire ni l'un ni l'autre.

Notion à découvrir : Notion : invariant et séparation des responsabilités.

Partie B — Rendre les actions cohérentes

Exercice 4 — Déplacer sans terminer l’activation

Travail demandé. Adapter deplacerUnite : après un déplacement accepté, placer l’unité dans l’état DEPLACEE plutôt que TERMINEE. Refuser une nouvelle tentative de déplacement si elle a déjà bougé. Conserver les contrôles de case existante, case libre et distance maximale de 3.

Contrainte. Réutiliser les vérifications préparées à l’exercice 3. Un déplacement refusé ne modifie ni les coordonnées ni l’état.

Validation. Premier déplacement valide : accepté, nouvel état DEPLACEE. Deuxième déplacement dans la même activation : refusé, position inchangée. Tentative initiale invalide : refusée, état toujours DISPONIBLE.

Exercice 5 — Attaquer après un déplacement

Travail demandé. Adapter attaquer pour autoriser un Epeiste vivant à attaquer s’il est DISPONIBLE ou DEPLACEE. Terminer son activation uniquement si l’attaque a lieu. Tester Chrom après un déplacement valide, puis tenter une deuxième attaque.

Contrainte. Réutiliser attaqueRapide et les autorisations centralisées. Ne pas recalculer les dégâts dans Partie ni permettre une attaque depuis TERMINEE.

Validation. Chrom se déplace, puis son attaque est acceptée ; Vaike perd 8 PV et Chrom passe à TERMINEE. Une nouvelle attaque est refusée et Vaike ne perd aucun PV supplémentaire.

Exercice 6 — Soigner après un déplacement

Travail demandé. Adapter soigner pour autoriser une unité implémentant Soigneur à soigner après s’être déplacée. Tester Lissa après un déplacement, puis une deuxième tentative de soin.

Contrainte. Conserver l’interface Soigneur et la méthode soignerCible. Le soin refusé ne modifie pas les PV ni l’état.

Validation. Lissa se déplace (DEPLACEE), soigne une fois (TERMINEE), puis son deuxième soin est refusé. Un personnage non soigneur ne peut toujours pas soigner.

Partie C — Fin d’activation et tours

Exercice 7 — Attendre depuis les deux états actifs

Travail demandé. Adapter attendre pour permettre à une unité DISPONIBLE ou DEPLACEE de terminer volontairement son activation.

Contrainte. Réutiliser la logique d’autorisation des actions ; ne pas modifier les PV ou la position. Une attente refusée ne change rien.

Validation. Une unité DISPONIBLE peut attendre ; une unité DEPLACEE peut attendre ; dans les deux cas elle devient TERMINEE. Une deuxième attente est refusée.

Exercice 8 — Fin de phase et nouveau tour

Travail demandé. Vérifier que phaseTerminee ne considère la phase terminée que lorsque toutes les unités de l’équipe sont TERMINEE. Adapter ou conserver reinitialiserEtat pour rendre DISPONIBLE uniquement les unités vivantes au début d’un nouveau tour.

Contrainte. Une unité DEPLACEE ne doit pas suffire à terminer la phase. Le nouvel état ne doit jamais restaurer des PV ni réanimer une unité vaincue.

Validation. Avec Chrom TERMINEE, Frederick TERMINEE, Robin DEPLACEE et Lissa TERMINEE : phase alliée non terminée. Après attente de Robin : phase terminée. Après réinitialisation : unités vivantes DISPONIBLE ; les unités à 0 PV gardent leur état précédent.

Exercice 9 — Tester les transitions interdites

Travail demandé. Construire dans Main un ensemble de scénarios courts qui tentent volontairement des actions interdites : deuxième déplacement, attaque après attente, soin après attaque, déplacement à 0 PV et déplacement vers une case occupée.

Contrainte. Contrôler à la fois la valeur de retour, l’état de l’unité, sa position et les PV concernés. Utiliser des unités et/ou parties de test distinctes si nécessaire pour ne pas fausser les résultats.

Validation. Chaque tentative interdite est refusée sans effet secondaire. Un déplacement invalide depuis DISPONIBLE laisse l’unité DISPONIBLE. Une unité vaincue ne se déplace pas.

Notion à découvrir : Notion : cas de test négatifs et préconditions.

Partie D — Intégration

Exercice 10 — Simuler un tour tactique complet

Travail demandé. Réorganiser Main pour jouer un tour de démonstration lisible : afficher la carte ; déplacer Chrom sur une case libre puis attaquer Vaike ; faire déplacer Lissa puis soigner un allié blessé ; faire attendre Frederick et Robin ; vérifier la fin de phase ; jouer l'action de Vaike ; vérifier la phase ennemie ; réinitialiser les états et réafficher la carte.

Contrainte. Conserver le comportement de la grille et des combats des TP précédents. Ne pas introduire JavaFX, IA, objets ou calcul de portée dans cet exercice. Éviter les anciens tests devenus inutiles au milieu de la simulation.

Validation. Les déplacements et actions réussis respectent les transitions ; les phases sont cohérentes ; les PV et positions affichés correspondent aux événements ; un nouveau tour remet uniquement les unités vivantes à DISPONIBLE.

Bilan attendu du TP13

À la fin du TP, le moteur n'assimile plus « s'être déplacé » à « avoir terminé son activation ». Le code exprime les transitions DISPONIBLE → DEPLACEE → TERMINEE, tout en autorisant DISPONIBLE → TERMINEE sans déplacement. Les refus d'action n'ont pas d'effet secondaire.

Auto-évaluation — questions à pouvoir expliquer

Pourquoi deux méthodes différentes peuvent-elles tester l'autorisation de se déplacer et d'agir ? Pourquoi un déplacement refusé ne doit-il pas changer l'état ? Quelle différence existe-t-il entre un état métier et un simple booléen ? Pourquoi conserver les contrôles dans Partie plutôt que les reproduire dans Main ?

Documentation à rechercher au besoin

Java enum ; méthodes et prédicats booléens ; switch sur une énumération (optionnel) ; tests de cas limites ; notion de finite state machine (FSM). Aucune bibliothèque externe requise.

Après chaque validation : effectuer un commit et un push GitHub avec un message du type « TP13 - Exercice N : ... ».